Part D.1 – Benchmarking the Stock GC Configurations

Design Optimization of Computing Systems – Assignment 1

1 Objective and Experimental Setup

The objective of D.1 is to benchmark the stock Full, Incremental, and Generational garbage collectors in Lua 5.5 and compare their execution cost. The experiment has three parts: Callgrind instruction analysis, workload scaling, and Linux `perf` hardware-counter analysis.

The supplied benchmark uses $m = 1000$, $n = 1000$, and `survive_every`=100. It therefore executes $1,000,000$ loop iterations. Each iteration creates an outer table and an inner table, giving approximately two million table allocations. Every 100th outer object is retained in `survivors`; most objects are therefore short-lived.

Executable	Configuration	Wrapper operation
`fullgc`	Full	`lua_gc(L,LUA_GCCOLLECT)`
`incrementalgc`	Incremental	`lua_gc(L,LUA_GCINC)`
`generationalgc`	Generational	`lua_gc(L,LUA_GCGEN)`
`nogc`	No GC	`lua_gc(L,LUA_GCSTOP)`

Table 1: Configurations used in the experiment.

An important implementation detail is that `LUA_GCCOLLECT` performs one full collection when the wrapper calls it; it does not establish a permanent "Full-GC mode". Thus, the Full and Incremental workload profiles are expected to be close.

2 D.1(a) Callgrind Analysis

Callgrind reports executed instructions using the `Ir` event. The inclusive instruction cost attributed to `luaC_step` is used as the high-level estimate of GC instruction consumption:

$$\text{GC share} = \frac{\text{inclusive instructions of } \texttt{luaC_step}}{\text{total executed instructions}} \times 100.$$

Nested functions such as `singlestep`, `propagatemark`, `atomic`, and `sweepgen` are not added separately because their inclusive costs are already contained in their parent call paths.

2.1 Original Workload: $m = 1000, n = 1000$

Configuration	Total `Ir`	Approx. GC instructions	GC share
Full	3,710,041,444	873.7 M	23.55%
Incremental	3,700,133,914	868.1 M	23.46%
Generational	3,574,116,509	748.1 M	20.93%

Table 2: Callgrind results for the original workload.

Full and Incremental GC are almost identical in this experiment. The Full wrapper performs its explicit collection before the benchmark, after which the workload continues normally. This explains why the measured workload profile remains close to the incremental profile.

Generational GC has the lowest measured GC instruction share. Its `luaC_step` share is 2.62 percentage points below Full GC and 2.53 percentage points below Incremental GC. The result is consistent with the benchmark because most newly allocated objects become unreachable quickly while only a smaller fraction survives.

2.2 Callgrind Behaviour

The Full and Incremental profiles contain large numbers of `singlestep` and `propagatemark` calls. For Full GC, `singlestep` occurs about 1.168 million times and `propagatemark` about 1.018 million times. The corresponding Incremental counts are about 1.159 million and 1.010 million.

The Generational profile is different. It has only about 86,815 `singlestep` calls and 148,334 `propagatemark` calls. The `sweepgen` path accounts for 16.65% inclusive cost. Thus, the lower GC instruction share is supported by a change in the actual GC execution path, not only by the final percentage.

3 D.1(b) Effect of Workload Size

The test was repeated with the three parameter sets specified in the assignment:

$$(m, n) = (100, 100), \quad (500, 100), \quad (5000, 100).$$

Workload	Full	Incremental	Generational
100×100	22.48%	22.13%	20.19%
500×100	23.02%	23.25%	20.67%
5000×100	23.38%	23.36%	20.84%

Table 3: GC instruction share for different workload sizes.

Workload	Full	Incremental	Generational
100×100	37,413,534	37,464,582	36,046,086
500×100	185,218,015	185,292,908	178,467,919
5000×100	1,858,691,647	1,856,319,711	1,784,962,952

Table 4: Total Callgrind instructions for different workload sizes.

The GC instruction share increases slightly as the workload grows: Full rises from 22.48% to 23.38%, Incremental from 22.13% to 23.36%, and Generational from 20.19% to 20.84%. The increase is not linear because both application instructions and GC instructions increase with workload size.

Generational GC remains the lowest for all three parameter sets. At 5000×100, Full and Incremental are almost identical (23.38% versus 23.36%). The number of allocated tables also scales with workload size, approximately 20,000, 100,000, and 1,000,000 tables for the three tested configurations.

4 D.1(c) `perf` Hardware Counter Analysis

The original $m = 1000, n = 1000$ workload was measured on native Linux using Linux `perf`. The measured events were branches, branch misses, cache references, cache misses, page faults, instructions, and cycles. IPC was calculated as

$$IPC = \frac{\text{Instructions}}{\text{Cycles}}.$$

Metric	Full	Incremental	Generational	No GC
Branch misses	83,559	81,319	93,655	40,890
Cache misses	43,652	43,982	29,758	463,476
Page faults	223	226	202	4,772
Instructions	372,688,637	372,665,431	358,850,300	412,773,951
Cycles	103,583,937	102,383,389	93,890,933	134,658,347
IPC	3.60	3.64	3.82	3.06

Table 5: `perf` results for the original workload.

4.1 Branch Misses

Branch misses indicate how often the processor's branch prediction was incorrect. This is relevant because GC code contains many conditional decisions involving GC states, object properties, marking, and sweeping.

Generational GC has the highest branch-miss count in this run (93,655), so its control flow is not uniformly easier to predict. No GC has the lowest count (40,890), but this does not imply better overall performance, because other costs are substantially higher.

4.2 Cache Misses

Cache misses indicate accesses that could not be served by the relevant cache level and therefore required more expensive memory access. They are important for this allocation-heavy workload because object creation, traversal, and reclamation repeatedly access dynamically allocated memory.

Generational GC has the lowest cache-miss count among the GC-enabled configurations: 29,758 compared with 43,652 for Full and 43,982 for Incremental.

No GC has 463,476 cache misses, which is dramatically higher. Since objects are not reclaimed during the benchmark, memory usage grows and the workload creates greater memory-system pressure.

4.3 Page Faults

Page faults indicate accesses requiring page-level memory management. They are relevant because continuous allocation can increase the amount of memory touched during the benchmark.

Generational GC has the lowest page-fault count among the GC-enabled configurations, at 202. Full and Incremental have 223 and 226, respectively. No GC has 4,772 page faults, consistent with increased memory pressure when allocated objects are not reclaimed during the run.

4.4 Instructions, Cycles and IPC

Generational GC executes the fewest instructions and cycles among the three GC-enabled configurations and has the highest IPC:

$$\text{Full} = 3.60, \qquad \text{Incremental} = 3.64, \qquad \text{Generational} = 3.82, \qquad \text{No GC} = 3.06.$$

The higher IPC for Generational GC means that more useful instructions are retired per processor cycle in this run. No GC has the highest instruction and cycle counts and the lowest IPC.

5 No-GC Baseline and Overall Interpretation

The No-GC configuration is an important baseline because simply removing GC work does not necessarily make an allocation-heavy program faster. With `LUA_GCSTOP`, objects are not reclaimed during the testbench, so memory usage and memory-system pressure increase.

In this run, No GC has the highest instruction count (412.77 million), highest cycle count (134.66 million), highest cache miss count (463,476), and highest page-fault count (4,772). It also has the lowest IPC (3.06). Although it has fewer branch misses, that single metric does not outweigh the much larger memory-related costs.

Across the GC-enabled configurations, Generational GC gives the most favorable measured results for this benchmark: the lowest Callgrind GC instruction share, the lowest instruction and cycle counts, the lowest cache misses and page faults, and the highest IPC. Full and Incremental remain very similar.

These conclusions are specific to the supplied allocation-heavy benchmark. A different workload with a different allocation rate, object-lifetime distribution, mutation pattern, or memory-access behaviour could produce different results.

6 Conclusion

The three required D.1 experiments give a consistent picture.

For the original workload, the GC instruction shares are

$$\boxed{\text{Full} = 23.55\%, \quad \text{Incremental} = 23.46\%, \quad \text{Generational} = 20.93\%}$$

Generational GC remains the lowest across all tested workload sizes. The `perf` results also show fewer instructions and cycles, fewer cache misses and page faults, and higher IPC for Generational GC in the measured run.

The No-GC experiment shows why disabling collection is not automatically beneficial for a program that continuously allocates objects: the memory system becomes a larger source of cost.

The Callgrind graphs, raw Callgrind outputs, and `perf` terminal outputs are submitted separately as supporting experimental evidence, as required by D.1.